

CMU-CS 462: Software Measurement and Analysis

Spring 2025-2026

Software Test Metrics

Nguyen Duc Man

E: mannd@duytan.edu.vn

T: 0904235945

12/01/2026

1

Contents

1. Test concepts, definitions and techniques
2. Estimating number of test case
3. Allocating test times
4. Test case management
5. Decisions based on testing
6. Test coverage measurement
7. Software testability measurement
8. Remaining defects measurement



2

What is a Test Case?

- **Definition:** A *test case* is a partial specification of a run through the naming of its direct input variables and their values.
- **Better Definition:** A *test case* is an instance (or scenario) of a use-case composed of a set of test inputs, execution conditions and expected results.
- Prior to specifying test cases one should document the use-cases.

3

Types of Software Test

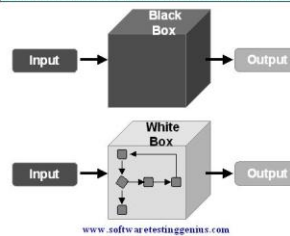
- **Certification Test:** Accept or reject (binary decision) an acquired component for a given target failure intensity.
- **Feature Test:** A single execution of an operation with interaction between operations minimized.
- **Load Test:** Testing with field use data and accounting for interactions among operations
- **Regression Test:** Feature tests after every build involving significant change, i.e., check whether a bug fix worked.

4

Basic Testing Techniques

- Two widely recognized testing methods:
 - **White Box Testing**
 - Reveal problems with the internal structure of a program
 - **Black Box Testing**
 - Assess how well a program meets its requirements

Comparison among Black-Box & White-Box Tests



www.softwarereviewing.com

5

1. White Box Testing

- Checks internal structure of a program
- Requires detailed knowledge of structure
- Common goal is **Path Coverage**
 - How many of the possible execution paths are actually tested?
 - Effectiveness often measured by the fraction of code exercised by test cases

6

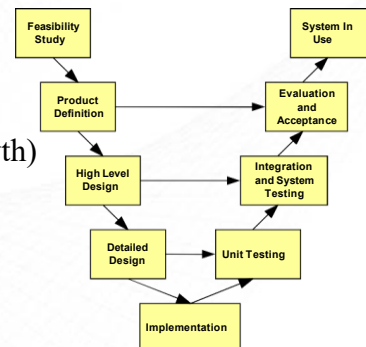
2. Black Box Testing

- Checks how well a program meets its requirements
 - Assumes that requirements are already validated
 - Look for missing or incorrect functionality
 - Exercise system with input for which the expected output is known
 - Various methods:
 - Performance, Stress, Reliability, Security testing

7

Testing Levels

- Testing occurs throughout software lifecycle:
 - Unit
 - Integration & System
 - Evaluation & Acceptance
 - Installation
 - Regression (Reliability Growth)



8

1. Unit Testing

- White box testing in a controlled test environment of a module in isolation from others
 - Unit is a function or small library
 - Small enough to test thoroughly
 - Exercises one unit in isolation from others
 - Controlled test environment
 - White Box

9

2. Integration Testing

- Units are combined and module is exercised
- Focus is on the interfaces between units
- White Box with some Black Box
- Three main approaches:
 - Top-down
 - Bottom-up
 - Big Bang

10

Integration Testing: Top-Down

- The control program is tested first
- Modules are integrated one at a time
- Major emphasis is on interface testing
 - Interface errors are discovered early
 - Forms a basic early prototype
 - Test stubs are needed
 - Errors in low levels are found late

11

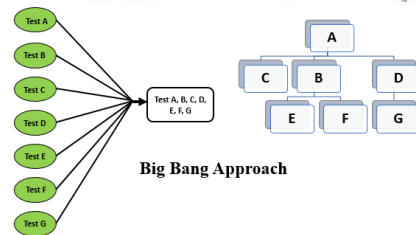
Integration Testing: Bottom-Up

- Modules integrated in clusters as desired
- Shows feasibility of modules early on
- Emphasis on functionality and performance
 - Usually, test stubs are not needed
 - Errors in critical modules are found early
 - Many modules must be integrated before a working program is available
 - Interface errors are discovered late

12

Integration Testing: 'Big Bang'

- Units completed and tested independently
- Integrated all at once
 - Quick and cheap (no stubs, drivers)
 - Errors:
 - Discovered later
 - More are found
 - More expensive to repair
- Most commonly used approach



13

3. External Function Test

- Black Box test
- Verify the system correctly implements specified functions
- Sometimes known as an *Alpha* test
- In-house testers mimic the end use of the system

14

4. System Test

- More robust version of the external test
- Difference is the test platform
- Environment reflects end use
 - Includes hardware, database size, system complexity, external factors
- Can more accurately test nonfunctional system requirements (performance, security etc.)

15

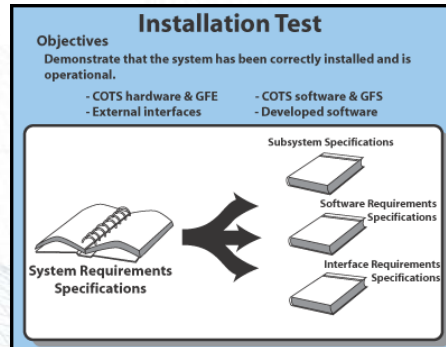
5. Acceptance Testing

- Also known as *Beta* testing
- Completed system tested by end users
- More realistic test usage than 'system' phase
- Validates system with user expectations
- Determine if the system is ready for deployment

16

6. Installation Testing

- The testing of full, partial, or upgrade install/uninstall processes
- Not well documented in the literature



17

7. Regression Testing

- Tests modified software
- Verify changes are correct and do not adversely affect other system components
- Selects existing test cases that are deemed necessary to validate the modification
- With bug fixes, four things can happen
 - Fix a bug; add a new bug; damage program structure; damage program integrity
 - Three of them are unwanted

18

Estimate No. of Test Cases

How many test cases do we need?

- Affected by two factors: **time** and **cost**
- **Method:**
 - Compute the number of test cases for the given
 - Time:

$$\frac{(\text{available time} \times \text{available staff})}{(\text{average time to prepare a test case})}$$
 - Cost:

$$(\text{available budget}) / (\text{average preparation cost per test case})$$
 - Select the minimum number of the two

19

Example

- The development budget for a project is \$4 million and %10 of which can be spent on preparing test cases. Each test case costs \$250 or 4 hours to prepare. The duration of project is set to be 25 weeks (of 40 hours each) and a staff of 5 is assigned to prepare the test cases. How many test cases should be prepared?
- From cost point of view:
- From time point of view:

20

How to Specify Test Cases?

- For a given operation, quantify the value of direct input variables based on *levels* for which the same behavior will be expected.
- A test case for a given operation will be specified using a combination of levels of direct input variables.
- If the number of combinations exceed the number of test cases assigned to that operation only a portion of combinations must be selected to represent all.

21

Example: Test Case

- Suppose that we are going to build test cases for an ATM system. The operation that we would like to test is “*Withdraw from Checking account*”. Suppose that an input variable named “**Withdraw_amount**” with the following specification is given.

Variable name/ definition	Specification
Name: Withdraw_amount	1. 3 digit numbers are accepted (withdrawal only between 100\$ and 1,000\$ cash)
Definition: The amount of cash that can be withdrawn in a single transaction	2. The number cannot start with 0
	3. The rightmost digit must be 0 (10\$, 20\$, 50\$ and 100\$ bills only)
	4. 4 digit numbers, only 1000 is acceptable
	5. Any other number of digits is not acceptable

22

Example: Test Case (Cont'd)

- Specify the “minimum” set of test-cases to test the operation for this variable. Note that we should avoid redundant test cases.

Case	Value of the variable “Withdraw_amount”	Criteria (pass, fail)
1	<i>i</i> where $0 \leq i \leq 9$	fail
2	<i>ij</i> where $0 \leq i \leq 9$ and $0 \leq j \leq 9$	fail
3		
4		
5		
6		
7		

23

How to Create Test Cases?

- Test case creation based on equivalent classes.
 - Prepare only one test case for the representative class.
- Test case creation based on boundary conditions.
 - Programs that fail with non-boundary values fail at the boundaries, too.
- Test case creation based on visible state transitions.

24

1. Equivalence Classes

A group of tests cases are “equivalent” if:

- They all test the same operation.
- If one test case can catch a bug, the others will probably do.
- If one test case does not catch a bug, the others probably will not do.
- They involve the same input variables.
- They all affect the same output variable.

25

Example: Equivalent Test Cases

A test case that covers case 2 will also cover case 1. Therefore, one test case based on 2 is enough to check both. Note that case 1 does not cover both.

Case	Value of the variable “Withdraw amount”	Criteria (pass, fail)
1	i where $0 \leq i \leq 9$	fail
2	ij where $0 \leq i \leq 9$ and $0 \leq j \leq 9$	fail
3		
4		
5		
6		
7		

26

2. Boundary Conditions

- Check if the boundary conditions for variables are set correctly.
- Check if the inequality boundary conditions can be changed to equality or not.
- Check if the counter input variables allow departure from the loop correctly or not.
- etc.

27

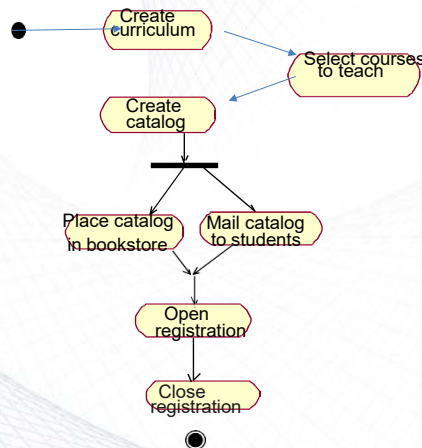
3. Visible State Transitions

- Every interaction with the program (i.e., setting an input variable, selecting a menu item, etc.) makes the program state move to another state (using UML *activity* or *statechart* diagram).
 - Test all paths that are likely to be followed by ordinary users under normal conditions (activity diagram).
 - Test any setting in one place whose effects are suspected to be propagated to elsewhere in the program (statechart diagram).
 - Test a few other random paths through the program.

28

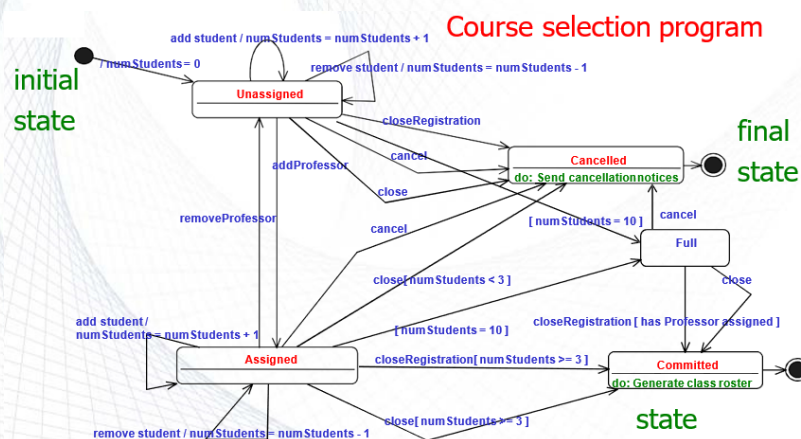
Example: Activity Diagram

- An activity diagram shows the flow of events within the use-case.



29

Example: Statechart Diagram



30

Allocating Test Time

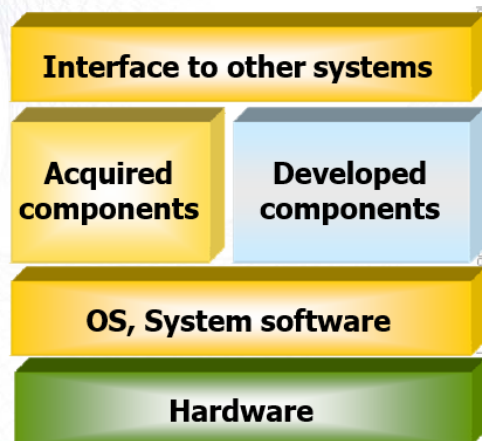
- How to manage test time?
- How to allocate test time among
- **system components** (e.g., acquired components, developed components)
- **test type** (e.g., certification test, feature test, load test, regression test) and
- **operation modes** (e.g., peak, prime and off hours modes)?

31

1. Test Time: Systems /1

Allocate time to interface to other systems based on estimated risk.

- Allocate less than 10% of the remaining time to certification test of acquired components.
- Allocate time to the developed components based on their significance.



32

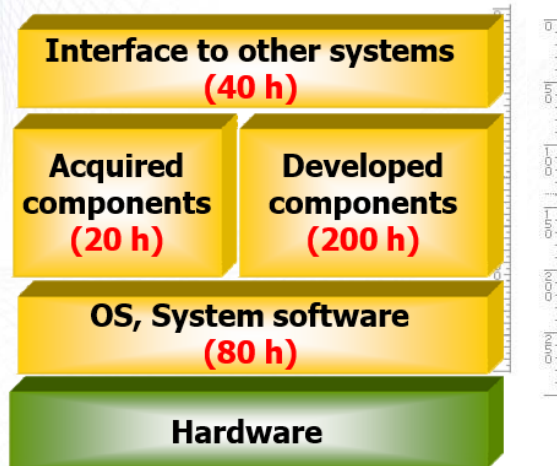
1. Test Time: Systems /2

Example:

- Total test time: 340 h (8.5 weeks)

Associated System Components 220 h

- Test case proportion:
 - Associated system components: 64.7%
 - Operating system component: 23.5%



33

2. Test Time: Test Type

- For each developed system components during reliability growth test:
 - for all new test cases: Allocate time to *feature test* (first release)
 - for all new test cases: Allocate time to *regression test* (subsequent releases)
- In this way testing all critical new operations will be guaranteed.
- **Example:** set aside 20 hours for feature test.
- The remaining time goes to *load test*.

34

3. Test Time: Load Test

- Allocate time for load test based on the proportion of the occurrences in operational modes
- **Example:** Web-based data transaction system

Operational mode	Proportion of transactions	Operational mode	Test Time (h)		
			Interface	Product	OS, etc.
Peak hours	0.1	Peak hours	4	18	8
Prime hours	0.7	Prime hours	28	126	56
Off hours	0.2	Off hours	8	36	16
		Total (h)	40	180	80

The remaining 20 hours goes to feature and/or regression tests.

35

Test Cases Management

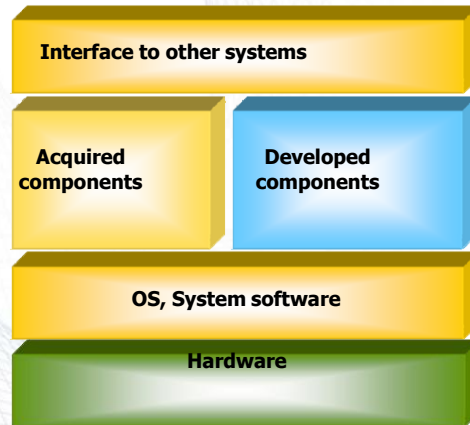
The procedure for preparing test cases involves:

1. Estimate the number of new test cases needed for the current release
2. Allocate the number of new test cases among the subsystems to be tested (system level)
3. Allocate the number of new test cases for each subsystem among its new operations (operation level)
4. Specify the new test cases
5. Adding the new test cases to the ones already available (may be from a previous release)

36

Test Case Allocation: System

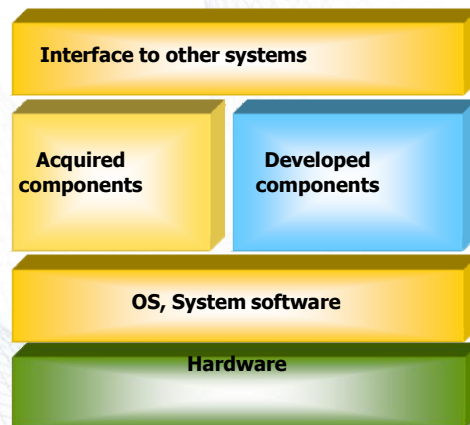
Allocate the bulk of the test cases to the developed product itself.



37

Test Case Allocation: System

Test Case Allocation: Operations



38

Example /1

- Total number of test cases: 500
- First release: All test cases are new.
- Suppose that we have one critical operation. Assign 2 test cases to it.
- Threshold occurrence probability: $0.5 / 500 = 0.001$
- Suppose that the number of infrequent operations with occurrence probabilities below threshold is 2.
- Assign 1 test case to each infrequent operation.
- Distribute the remaining $500 - (2+2) = 496$ test cases among the rest of operations based on their occurrence probabilities.

39

Example contd. /2

- Example: Occurrence probabilities for normal operation mode.

Operation	Occurrence probability
Process voice call, no pager, answer	0.18
Process voice call, no pager, no answer	0.17
Process voice call, pager, answer	0.17
Process fax call	0.15
Process voice call, pager, answer on page	0.12
Process voice call, pager, no answer on page	0.10
Phone number entry	0.10
Audit section of phone number database	0.009
Add subscriber	0.0005
Delete subscriber	0.0005
Recover from hardware failure	0.000001
Total	1.0

Infrequent operations
below threshold

Table from Musa's Book

41

40

Example contd. /3

- Example: Occurrence probabilities for normal operation mode.

Operation	Number of test cases
Process voice call, no pager, answer	89
Process voice call, no pager, no answer	84
Process voice call, pager, answer	84
Process fax call	74
Process voice call, pager, answer on page	59
Process voice call, pager, no answer	50
Phone number entry	50
Audit phone number database	4
Add subscriber	1
Delete subscriber	1
Recover from hardware failure	2

Divided based on occurrence probabilities

Infrequent operations below threshold

Critical operation

Table from Musa's Book

41

Discuss in group of 4 students



Software Testing Metrics and KPIs?

42

References and Assignment

1. Lincoln, C. R. (2005). Best Practices in Software Measurement: How to Use Metrics to Improve Project and Process Performance. *Software Quality Professional*, 7(3), 48. (pp. 95-131, pp.219-300)

2. Watch the below video and answer the following question:
<https://youtu.be/OZLMKTjPr6g>

Summarize the contents of the lecture in the video!

3. <https://www.softwaretestingmaterial.com/test-metrics/>
<https://www.tricentis.com/blog/64-essential-testing-metrics-for-measuring-quality-assurance-success/>

43

Bài tập

Scenario:

You are managing the testing of a **Mobile Banking Application**. You have been tasked with calculating the **Test Metrics** for the first phase of testing, which includes 100 test cases, 80 of which passed, and 20 of which failed.

Your task is to calculate the following metrics:

1. **Test Execution Rate**
2. **Defect Density**
3. **Pass Rate**
4. **Defect Discovery Rate**

44